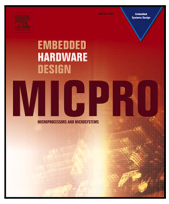
Microprocessors and Microsystems 112 (2025) 105132



Contents lists available at ScienceDirect

Microprocessors and Microsystems

journal homepage: www.elsevier.com/locate/micpro





Design and implementation of a synchronous Hardware Performance Monitor for a RISC-V space-oriented processor

Miguel Jiménez Arribas *, Agustín Martínez Hellín, Manuel Prieto Mateo, Iván Gamino del Río, Andrea Fernández Gallego, Óscar Rodríguez Polo, Antonio da Silva, Pablo Parra, Sebastián Sánchez

Space Research Group, Department of Automatics, University of Alcalá, Alcalá de Henares, 28805, Madrid, Spain

ARTICLE INFO

Keywords:
Performance counters
Performance measuring unit
RISC-V
Computing architecture
On-board computing

ABSTRACT

The ability to collect statistics about the execution of a program within a CPU is of the utmost importance across all fields of computing since it allows characterizing the timing performance of a program. This capability is even more relevant in safety-critical software systems, where it is mandatory to analyze the software timing requirements to ensure the correct operation of the programs. Moreover, in order to properly evaluate and verify the extra-functional properties of these systems, besides timing performance, there are many other statistics available on a CPU, such as those associated with its resource utilization. In this paper, we showcase a Performance Measurement Unit (PMU), also known as a Hardware Performance Monitor (HPM), integrated into a RISC-V On-Board Computer (OBC) designed for space applications by our research group. The monitoring technique features a novel approach whereby the events triggered are not counted immediately but instead are propagated through the pipeline so that their annotation is synchronized with the executed instruction. Additionally, we also demonstrate the use of this PMU in a process to characterize the execution model of the processor. Finally, as an example of the statistics provided by the PMU, the results obtained running the CoreMark and Dhrystone benchmarks on the RISC-V OBC are shown.

1. Introduction

RISC-V is a CPU architecture which has been receiving a lot of attention in recent years thanks to its advantageous characteristics, namely being an open standard, its modular design, a prominent academic and open source community, and having the freedom to adapt and use it in vastly different fields and applications, from embedded systems to large-scale computing [1]. This has also sparked the interest of the space industry, both from NASA in the US with the HPSC from JPL [2], and in Europe, where there have been some works towards the adoption of RISC-V as the new standard architecture for space [3–5].

In the last three decades, the Space Research Group (SRG) of the University of Alcalá has participated in the development of flight software and hardware for different space missions [6–9]. This experience has motivated different research works oriented to facilitate the fulfillment of the reliability requirements demanded by space missions [10–13]. With the same approach, this is why of the whole RISC-V architecture we have focused specially on what allows an implementation to assess its correct behavior and performance, namely for now, its tracing mechanisms [14] and the performance counters.

This article focuses on the latter mechanism. It proposes a solution that facilitates behavior characterization and performance measurement of software deployed on RISC-V based on-board computers (OBCs). Specifically, this work shows a performance measurement (or monitoring) unit (PMU) or, in RISC-V nomenclature, a hardware performance monitor (HPM). Other terms, such as statistical unit or performance counters, are also often used in the literature for this type of unit, so from now on, any of these names will be used interchangeably.

This PMU has been integrated into a RISC-V soft-core on-board processor for FPGA with a segmented pipeline targeting space applications. The PMU supports the events standardized by the RISC-V specification [15] and allows its extension with additional events both during synthesis and in execution time. It presents a very extensible design, capable of accepting new events with a minimal development cost. Additionally, it offers the advantage of extracting timing information from the measured events, since the counting process is synchronous with the execution of the instructions. This improves the value of the PMU, as it is now possible to see when each event occurs and how they relate to each other.

* Corresponding author.
E-mail address: miguel.jimeneza@uah.es (M. Jiménez Arribas).

https://doi.org/10.1016/j.micpro.2024.105132
Received 9 August 2023; Received in revised form 6 April 2024; Accepted 11 December 2024
Available online 17 December 2024
0141-9331/© 2024 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

The primary use case that benefits from the implementation of a PMU is, during the development process, to facilitate the design, development, and debugging of the software, and even of the hardware itself; since with it we can collect the properties of the architecture and correct or optimize its behavior. Typically, after launch, having passed the commissioning state, the HPM can be disabled, to reduce the footprint of the on-board computer (OBC). However, as we shall see throughout the rest of this article, it could be argued that in this case, the increase in resource and power utilization with the PMU enabled is sufficiently small that, depending on the circumstances, it could even be considered advantageous to keep it activated in order to debug problems during flight.

The remainder of this paper is structured as follows: first, in Section 2, a brief overview on the state of the art is outlined. Then, in Section 3, the main characteristics of the PMU design and its implementation details are discussed, along with the motivation and rationale of their selection. Here, a brief example of operation is also described. Subsequently, in Section 4, using the data obtained with the PMU, as evidence of its usefulness, the execution timing behavior of our processor is characterized. Next, in Section 5, the results obtained with the HPM during the execution of different benchmarks used to test the correct operation of the proposed design are presented. Additionally, the results are compared with those of another processor, to externally validate the design and to illustrate the enhancements with other PMU implementations. Moreover, the resource utilization and power measurements of this implementation, in comparison with the same developed OBC but without the hardware performance monitor enabled, are also shown. Finally, in Section 6, the conclusions are drawn.

2. Related works

As stated in the introduction, there has been a significant increase in interest in utilizing the RISC-V architecture for space applications in Europe over the last 5 years [3]. This attention has now escalated beyond purely academic interest due to its advantageous features and now includes economic incentives. For instance, Cobham Gaisler has developed an alternative to the standard OBC in the European space industry, LEON, with a new processor based on RISC-V called NOEL-V [16]. Furthermore, the European Union has created various projects within its H2020 framework for the development of the necessary infrastructure for the maturation of this architecture in the European safety-critical and space landscape [17,18].

In addition to its favorable economic outlook, the appeal of RISC-V is also based on its technical characteristics. Specifically, in the area that concerns us, RISC-V allows quite a scalable design for the PMU [15,19]. It includes 32 counters out of which only 3 have a standardized purpose, with the other 29 capable of being configured to measure any event needed, both at runtime and during hardware synthesis. These 29 counters can be configured via control registers, each of which allows at least 2^{32} selectable events for a single counter, a value that exceeds any practical implementation, producing considerable design flexibility.

Regarding other architectures, current multipurpose processors like Intel's [20] or ARM's [21] include proprietary PMUs. These units still primarily only measure internal microprocessor events, not system-level tasks, or other software constructs [22], even though they are much more complex than current embedded architectures like, for example, LEON's [23,24].

In general, most PMU designs have two main mechanisms, an event detector, and the event counters [25]. With this type of system, it is possible to create performance profiles with which to analyze the performance and behavior of the processor and the program. These are called time-based profiles and event-based profiles [25]. Nevertheless, with this design of PMU it is difficult to obtain timing data on when each event occurred and how these events are related to each other [25,26]. This is the reason why the design presented in this paper synchronizes its counting process with the execution of instructions, as will be discussed in more detail in the following section.

Finally, in addition to the previously mentioned specification, some further works have been conducted regarding the HPM of RISC-V. Firstly, it was analyzed regarding its use with the currently available open-source tools. With this, recommendations were made on how the spec and surrounding software should be improved [27]. And secondly, other works have deepened the flexibility of the specification, improving its task awareness and in general the ease of configuration [28,29].

With all of the aforementioned academic work regarding the HPM of RISC-V, and the industry's adaptation of the Linux kernel and other open-source software infrastructure to RISC-V, an updated standard [30] has been produced. This update provides clarification and some pending functionality which has been long requested, mainly the possibility of launching interrupts when one of the counters suffers an overflow, and the filtering of events depending on the privilege level. Still, even after these updates, we believe that our work is still of interest as no other work that we have found explains in detail the integration and implementation of the PMU within a pipelined architecture. In addition, our design also benefits from the features outlined above, as will be further explained in the next section.

3. Development

The implementation presented here was born out of the interest of our research group in finding ways to improve the observability, testability and reliability of processors for space applications.

We have gathered considerable experience in this area, having been in charge of creating the hardware and software for the ICU [9,11] for the EPD instrument [31,32] inside the Solar Orbiter mission. In the last 3 years, we have been working on a RISC-V processor where we could instill our experience. Therefore, as mentioned previously in Section 1, we have been focusing on tracing [14] and performance measuring, to be able to gather the behavior of the architecture and ease the development and debugging process of software.

Looking specifically at the PMU, this design was specially created with three goals in mind: firstly, to allow the monitoring of events without interfering with the behavior of the existing pipeline, and thus also transparent to the execution time; secondly, to synchronize the event count with the execution of each instruction so that the maximum observability could be achieved; and thirdly, to support the extensibility of the events that the specification proposes, so that new ones could be added with minimal development cost.

Hence, in the following subsections the development of this PMU is presented. Initially, the baseline processor is introduced, which served as the starting point for the proposed design. Then, in the following subsection, the PMU design and its motivations are detailed. Subsequently, in the next two subsections, its configuration, and some of its implementation details are shown based on a RISC-V architecture [15] pipelined OBC. Additionally, a specific example of operation across the entire pipeline is described in the final subsection.

It is important to clarify that while the objective of developing the PMU is to support reliability, its primary function is to provide developers with the essential tools for monitoring and analyzing the processor's behavior. As such, the PMU serves as a foundational component, offering insights into execution patterns and performance metrics vital for diagnosing potential issues and optimizing software performance. However, it is crucial to note that the PMU, in isolation, does not directly deliver reliability. Achieving reliability in processor systems typically necessitates a multifaceted approach, encompassing fault tolerance mechanisms, error correction techniques, and rigorous testing protocols. Thus, while the PMU lays the groundwork, contributing significantly to the development process by enhancing observability and testability, its immediate impact on reliability is indirect, being part of a broader strategy aimed at cultivating reliable and robust processor

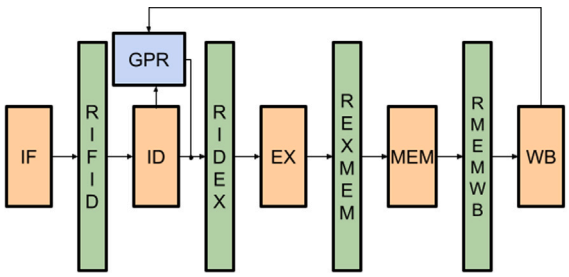


Fig. 1. Baseline pipeline structure. The 5 stages mentioned are shown in orange, while in green, the inter-stage registers that synchronize the data transition between them are found. Finally, in blue, other functional units are shown. Specifically, the placement of the general-purpose register file can be seen. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

systems for space applications. Nonetheless, it could be argued that, in comparison with other PMUs, the proposed design offers superior reliability, a topic extensively examined in Section 3.2.

Lastly, even though for this paper the focus has been on the RISC-V architecture, since the basis of the PMU presented here has been what is defined in its specification, it should be noted that the actual design principles really are ISA-agnostic and could therefore be applied to other CPU architectures interchangeably, as will be demonstrated in the next subsections.

3.1. Baseline processor

Before focusing on the design of the PMU a brief description of the baseline pipeline from where the design evolved will be explained. In Fig. 1 a schematic of this baseline architecture can be seen. In it, a typical 5-stage pipeline design can be observed, between which the inter-stage registers manage the propagation of the results, enabling the synchronization of the pipeline. In addition, although for simplicity and ease of understanding they are not represented in the figure, there are some additional components in charge of other functions of the pipeline, e.g., a data forwarding unit (FU) for overcoming data hazards and some additional units for memory accesses and exception flow i.e., traps, etc.

Briefly, the functionality of each stage is the following: the Instruction Fetch stage (IF) controls the insertion of instructions into the pipeline from the instruction memory; the Instruction Decodification stage (ID) recognizes each instruction, detects data hazards, and supplies the appropriate control signals to the remaining stages; the Execution stage (EX) calculates the results needed for the following stages, for instance, it is also here where, in case of branch instructions, it decides whether or not to take them and resolves the jump address. Next, the Memory stage (MEM) is in charge of storing and loading data to and from data memory. And finally, the Write-Back stage (WB): oversees which data, if at all, is stored back in the GPRs, be it the result from the EX stage or the value gathered from memory in the MEM stage.

Gamino et al. in [14] give a more detailed look of the whole processor and its distinctive features. From this point onwards the focus will solely be on the design of the PMU. Nevertheless, it is important to remember the segmented nature of the processor, as this is critical for the comprehension of the design.

3.2. PMU design: motivating factors and proposed solution

The PMU design depicted in this paper comprises two main innovative approaches: its decentralized triggering system and its synchronized counting process. The first approach is provided by the most important data structure of the PMU design i.e., "*Triggered_events*", which stores whether each of the supported events has been detected on any of the units composing the pipeline, and, as shown in Figs. 2 and 4, it is chained through the pipeline until it reaches the Control and State Registers (CSRs) where the events are finally counted.

This could be considered a decentralized PMU design, and it has the advantage over centralized designs in that events are triggered at the pipeline stage where they arise, rather than feeding all the control signals to the counter unit and triggering the events in the same unit where the counting occurs. A fully centralized design is rare, as the amount of information needed to be fed would increase complexity exponentially. Nonetheless, hybrid designs are quite common, as can be seen on Gaisler's GR740 board [33–35].

Instead, the design proposed here is entirely decentralized. Therefore, events are triggered across the pipeline, in the same unit where they are detected, and then chained through each remaining pipeline stage, from the Instruction Fetch, i.e., the start of the pipeline, to the Write-Back, i.e., the final stage. This approach simplifies the detection logic, as shown in more detail in Fig. 3, decreasing its complexity, and thus, facilitates the extensibility of the events supported.

The other major contributing aspect of this design is the synchronization of the counting process by modifying the moment at which the count is performed. In other CPUs, in most cases it is impossible to accurately attribute an event to a specific instruction. The difficulty arises because events are counted within the monitoring unit immediately after they have been triggered. Consequently, if events occur at different stages of the pipeline, each event is counted at a distinct phase of its execution, without actually being synchronized with the completion of the instruction execution. As a result, precisely attributing an event to the instruction that generated it becomes challenging. Moreover, there is a risk of potentially counting an erroneous event which would have been canceled in subsequent stages.

A typical example of this is the event of instruction retirement. There are CPUs which increment this type of event the moment the instruction enters the execution stage, regardless of whether this instruction completes execution or if an error is encountered while still in the pipeline and an exception is thrown [36].

One possibility to solve this problem would be making the event detection logic more complex, as will be discussed in subsequent paragraphs, controlling all the possibilities of event cancellation in the detection logic. But this still would not solve the issue with counting on a different stage as when the instruction is finally committed to the register file, which can still pose significant challenges.

Another even more extreme example arises while utilizing the Sscofpmf extension [30], that allows the generation of interrupts when one of the PMU counters overflows. One could imagine a scenario where different subroutines were incrementing a specific PMU counter. The case may arise where instructions of multiple subroutines were found in the pipeline when an overflow interrupt of that counter is produced. Due to the previously mentioned problem, the latency within the CPU between triggering the event and executing the instructions, the program counter (PC) of the instruction delivered to the interrupt handler may not be the one that caused the event. In fact, the difference between them can vary by an unpredictable amount, as it could be the PC of any of the instructions of any of the subroutines which were incrementing the counter. One could argue, depending on the implementation of the detection logic, that for in-order processors the PC must belong to the instructions within the pipeline, but out-of-order processing exacerbates this problem. This discrepancy could result in faulty information being received by the interrupt handler, potentially leading to undefined behavior.

Importantly, these kinds of problems are possible with any type of event, depending on the design of the PMU, not only with those in the examples provided, and although they are more common in out-of-order processors, problems can also be found among in-order processors. The works [25] and especially, section 2.2 of [26], discuss more about the intricacies of synchronizing the event counting with the instruction retirement.

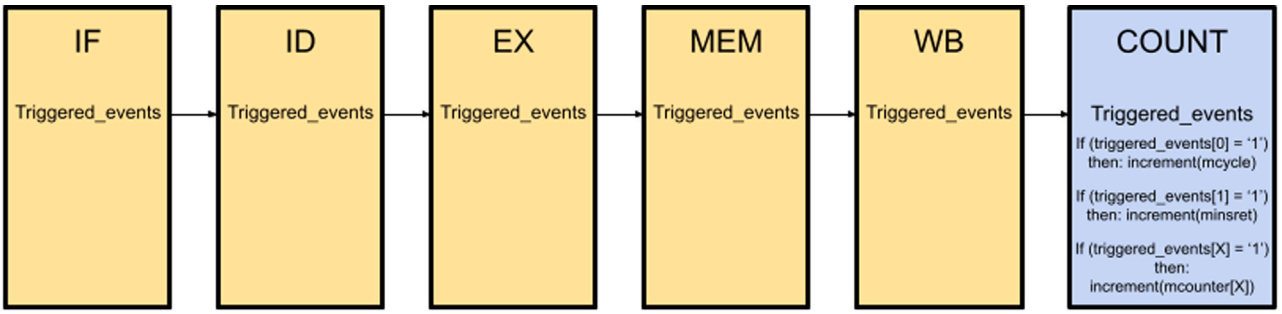


Fig. 2. This is a simplified abstraction that shows the coupling between each of the five stages of the pipeline and their corresponding inter-stage registers, hence the difference in color coding with the amalgamations in yellow. In addition, it also shows the *triggered_events* data structure and how it is monitored and chained through the pipeline arriving to the count module where the events are finally added up. This module is an auxiliary functional unit and therefore not an actual stage in the pipeline since it does not produce any effect in the execution of instructions, thus the blue color. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

This design aims to resolve these problems by instead storing trigger information within the pipeline, synchronizing event counting with instruction retirement. In Fig. 2, a simplified abstraction of the pipeline can be seen, where each rectangle represents the fusion between the stage and the inter-stage registers. This figure also shows that the counting of the events is set to only occur on the next cycle after the instruction has finally been written back to the registers, either the General-Purpose Registers (GPRs) or the CSRs.

While, at first glance, it might seem a mistake to account for events later than they happen, this is a side effect of the fact that the events can appear at any moment during the execution, as has been explained. For example, the case may arise that the retirement of an instruction is set to be counted and during the WB stage it is detected that the result cannot be written. Thus, the events triggered by this instruction need to be updated before being recorded on the PMU. Hence, it is necessary to wait until the completion of its execution, i.e., the next cycle after the write-back stage, to finally count all the events.

In general, there are various other ways that the event triggering could be synchronized. For example, Nam Ho et al. in [24,37] receive the event pulses and then it is the control logic within the PMU module itself what manages when the counters are finally incremented. Meanwhile, other more complex CPUs, like for example the ones which support Out Of Order and speculative execution [26,38], confront this problem via counting the events triggered immediately and waiting until the end of the execution in case any problem was encountered, to either commit the results, or instead, undo the produced events and any other type of side effects generated. Given the complexities and limitations of these latter approaches, they were deemed a hindrance and the simpler design with the considerations discussed in the preceding paragraphs was opted for, as any kind of benefit was outweighed by the drawbacks.

An in-depth examination of the integration of the design within the existing processor is illustrated in Fig. 3. This PMU design involves capturing signals for each event from each of their respective units and, rather than directly routing them to the counter unit as commonly practiced in other PMU designs, concatenating them through the pipeline. Considering this design, several noteworthy aspects emerge. Firstly, there is no timing penalty incurred during instruction execution, as the standard pipeline remains unaltered. The only aspect which is delayed is the arrival of events to the counter unit, but as explained previously this is deliberate, as this way each event can be attributed to the corresponding instruction which is completing execution. Secondly, although the size of the inter-stage registers obviously increases, as will be elaborated in more detail on Section 5.3, this difference is negligible, as it only increases one bit per tracked event per register. Therefore, the expansion in size is minimal in comparison with the value of the information provided. Thirdly, with respect to the critical path potentially impacting processor frequency, as depicted in Fig. 3, the modifications solely add changes in a parallel manner. There is

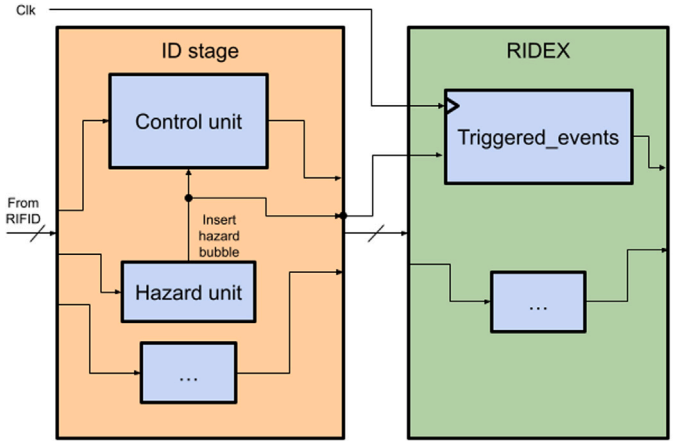


Fig. 3. Example illustrating the integration into the existing processor of the hazard event detection mechanism and its storage under the proposed design. As can be seen, the signal "Insert hazard bubble" was already necessary so that the control unit knows when to insert a bubble. Therefore, the PMU mechanism monitors existing signals within the design and registers them in the *triggered_events* data structure. Notably, this process occurs in parallel without introducing sequential logic, ensuring no timing penalty.

no additional sequential logic that could elongate the path the signals could take with more operations; instead the existing value is simply channeled into a new parallel register. This strategic inclusion of parallel registers for event tracking ensures that the signal processing speed remains unaffected. Consequently, the processor's ability to maintain its performance metrics, despite the augmented functionality of the PMU, remains intact.

Finally, regarding support of this PMU design for more complex architectures, it is pertinent to acknowledge that while the primary objective of this article is to present and validate the core functionality, some consideration has been directed towards potential enhancements for advanced architectures. As mentioned previously, one of the key aspects of the proposed design is that it is architecture agnostic. The performance monitoring hardware is only intrinsically linked with the pipeline, as it is here where it detects and chains the occurred events, rather than with the decodification and execution of instructions, i.e.: the ISA. As such, the development and behavior of this design in the context of more advanced microarchitectures would remain equivalent.

For instance, in pipelined microarchitectures featuring multi-cycle or even out-of-pipe functional units, like for example those common on multiplication or division operations [39], the instructions would still reach the MEM and WB stages, and therefore its inter-stage register. Then, once these instructions would arrive at this register, the events occurred during its execution would trigger the same way as already

described, and when the instructions were committed to the register file their corresponding events would be counted.

Similarly, in architectures employing superscalar or out-of-order execution, although their microarchitectures are more intricate, they still rely on inter-stage registers for signal synchronization, so the newer events from these architectures could be detected there. And although, in out-of-order processors execution may occur out of order, the actual commitment of instructions still adheres to architectural order using register renaming. Therefore, consideration should be given to associating events with instructions based on the real physical registers rather than the logical architectural ones during instruction commitment. Furthermore, regarding multi-core architectures, each core or hart, in the usual RISC-V terminology, would possess its own set of events and performance counters, enabling independent monitoring and analysis tailored to each core's activity.

Lastly, the treatment of cache memories has not been considered in this work. It is widely recognized that cache memories introduce inherent uncertainties in the pipeline flow, mainly due to their hit/miss ratio, cache memory configuration and software coding. These uncertainties can make the WCET difficult to estimate [40,41]. Therefore, since the primary objective of this paper is to characterize and validate the core functionality of the presented HPM for RISC-V, the inclusion of cache memories would have introduced additional complications and, as a first approximation, for the time being, they have not been implemented. Consequently, the usual cache hits and misses events are not yet supported, as detailed in the next subsection. Nevertheless, increasing processor performance stands as an imperative in the contemporary landscape, both for the old and for the new space paradigms. Hence, providing execution acceleration mechanisms (cache, multicore, etc.) support is one of the first improvements planned. For example, implementation strategies could mirror those employed in architectures like Intel's Nehalem, where caches are embedded into the pipeline [42], facilitating that events could be automatically detected and triggered in a process equivalent to what has already been explained. Thereby, requiring no modifications to the PMU design philosophy. In cases where support for caches beyond L1 is warranted, such as L2 or L3 caches, the fetch stage would assume responsibility for managing events generated by these caches via additional communication lines. Then, after an analysis to characterize the timing behavior of the cache, such as the one that can be seen in [40], these will be integrated into our OBC and the actual events supported by the PMU.

3.3. PMU configuration

In Fig. 4, the full PMU design can be appreciated. The figure shows how the PMU spans across the whole pipeline, with the events triggered in each stage being chained through the remaining stages and finally reaching the CSR unit. Furthermore, it also shows where the configuration registers are located in order to decide the functionality of the PMU and where the counters are incremented.

Once the PMU behavior has been described now these configuration registers will be explained. As mentioned earlier, this configuration could be changed at any moment, both during synthesis and at runtime. According to the scope of their function, these registers could be separated into general and specific configuration registers, and they are the same as those defined in the RISC-V specification [15]. On the one hand, the general configuration registers are *mcounteren* and *mcountinhibit*. The former decides whether each counter is accessible in both user and machine mode or, instead, they are only accessible in machine mode. Whereas the latter register inhibits the counting of the bit-selected counters. This can be useful in several ways, such as to reduce power consumption, or to allow access to the entire set of counters without their value changing.

Meanwhile, on the other hand, the specific configuration registers (*mhpmevent3 - 31*) oversee the selection of which event is monitored by which counter so that when triggered, an increment is fulfilled in the selected counter. The supported events can be found in Table 1. This table also depicts the aforementioned relationship between the counters and the event selected for each counter.

Table 1
Table showing the matching between each counter, its configuration register, all the currently supported events and which counter counts each event.

Counter	Configuration register	Programmed event
mcycle	–	HPM_EVENT_CYCLE
–	–	–
minstret	–	HPM_EVENT_INSTRET
mhpmcounter3	mhpmevent3	HPM_EVENT_EXCEPTION
mhpmcounter4	mhpmevent4	HPM_EVENT_EXT_INT
mhpmcounter5	mhpmevent5	HPM_EVENT_TIME_INT
mhpmcounter6	mhpmevent6	HPM_EVENT_BRANCH
mhpmcounter7	mhpmevent7	HPM_EVENT_BRANCH_NT
mhpmcounter8	mhpmevent8	HPM_EVENT_UNCOND_JUMP
mhpmcounter9	mhpmevent9	HPM_EVENT_HAZARD
mhpmcounter10	mhpmevent10	HPM_EVENT_MEM_ACCESS
mhpmcounter11	mhpmevent11	HPM_EVENT_LOAD
mhpmcounter12	mhpmevent12	HPM_EVENT_STORE
mhpmcounter13	mhpmevent13	HPM_EVENT_FETCH

Now a brief description of each event will be provided. First, the cycle event counts the number of cycles that have elapsed since its reset, and it is triggered and incremented on every rising edge for as long as its corresponding counter is not inhibited. On this note, outside the main unit of the PMU (as it is outside of its definition in the specification [15], though it can be accessed through it [19], hence the second row in Table 1), the OBC also holds a real-time counter. This register, currently, is implemented as a cycle counter of constant frequency, but without the possibility of being inhibited, thus acting as the real-time clock source for the whole core. However, the option remains that a suitable quartz crystal may be incorporated as the real-time RTC source in the future.

The next event is the execution of an instruction, also known as retirement in RISC-V terminology. This event occurs when the instruction finally completes the last stage, and the results are committed back to the register file. Up to this point, these two are the only events declared in the RISC-V specification; the rest are left platform specific.

In particular, for the platform presented in this paper, the PMU has been extended to support several other events, based on what has been deemed appropriate given the intent of this on-board processor to be for space applications.

Thus, the first additional supported events are for counting the number of exceptions and other types of traps encountered during execution, such as both timer and external interrupts. Secondly, branches, both taken and not taken, and unconditional jumps are also accounted for, triggered during the execution stage. Other instructions that merit accountability, as they often contribute to pipeline delays, are memory instructions, which generate an event for each type of operation: loads, stores and fetches. Moreover, there is a more general event to count when any instruction accesses memory through the MEM stage. Finally, hazards are monitored on the ID stage. It is worth noting that here the term 'hazard' specifically refers to those instances that cannot be resolved via the forwarding unit, necessitating the insertion of a bubble in the pipeline, as the rest are transparent and do not affect the CPI.

As it can be observed, each event that could modify the cycles per instruction (CPI) metric is accounted for; hence, this way, the behavior of the OBC can be characterized as will be shown on Section 4. It should be noted that not all events were supported from the start, but rather, as these tests were performed, additional events were progressively included, demonstrating the rapid and easy extensibility of the PMU.

3.4. Implementation details

Finally, besides the microarchitectural changes described above, to provide more depth to the modifications needed to integrate the PMU

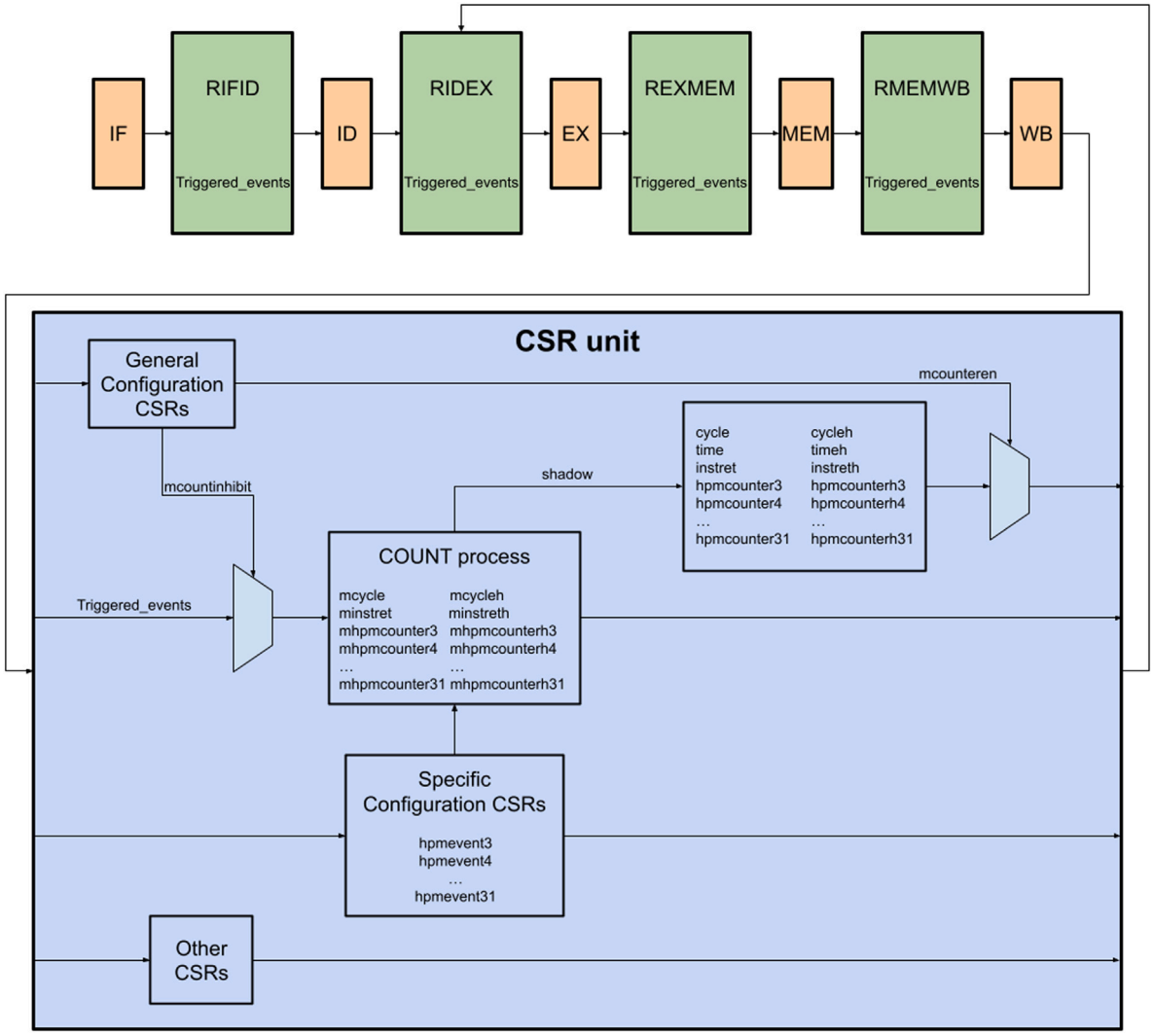


Fig. 4. Here the complete diagram of the PMU is shown. As has been explained, its reach spans throughout the entire pipeline by storing the monitored events in the *triggered_events* data structure, which ultimately arrives at the CSR unit. Then it is here where the behavior of the PMU is decided with its configuration registers, and where the performance counters are located and finally incremented.

inside of an already existing RISC-V OBC, the synchronism mechanisms for counting will be explained.

First, it must be noted that according to the RISC-V specification [15,19], the PMU is defined inside the Control and Status Registers (CSRs), and hence those must be supported. These registers have the particularity of needing to be accessed atomically to prevent the formation of race conditions during the configuration of the internal state of the CPU. Thus, the PMU counters also must be accessed as such.

The selected design for atomic access consisted of performing both reads and writes in the same clock cycle, one on each clock edge. Therefore, the sequence of accesses would be the following: initially, during the rising edge, the old value of the CSR is stored on an intermediary structure at the exit of the CSR module. Later, at the falling edge, two writes occur simultaneously on different components: the corresponding CSR is written with the value of the general-purpose source register (*rs1*), and the intermediate value with the former value of the CSR is stored in the destination GPR register (*rd*). A visual representation of this transaction can be seen in Fig. 5.

One more thing to note is that, in the case various accesses were made to the same CSR sequentially, for example with the sequence of instructions seen in Listing 1, new types of data hazards need to be detected during the ID stage.

Listing 1 Sequence of instructions that updates the *mepc* CSR after a trap. These instructions would cause two new CSR hazards. First, the second instruction would need to wait until the CSR instructions finishes its execution to update the value of the *t1* register in the WB stage, as the value cannot be forwarded due to the atomicity of the CSRs. And second, for the same reasons, the third instruction needs to wait until the second arrives to the WB stage, to avoid a writing the wrong value to the *mepc* CSR.

```
csrr t1, mepc
addi t1, t1, 4
csrw mepc, t1
```

Now, by inserting the PMU counters into the equation, there is one last additional transaction to complete during that same cycle in which a read and a write can be performed. In this instant, when the corresponding event to a specific counter has been triggered, a count must also be fulfilled. This event update also occurs on the rising edge.

However, a problem arises in this situation, since it is apparent that a write and an increment may have to be performed on the same counter. Here, 3 different scenarios might happen: first, if the write occurs in the cycle before the triggered event arrives at the count stage, the write occurs nominally and in the next cycle, once the counter is already updated after the write, the event count occurs. However, in

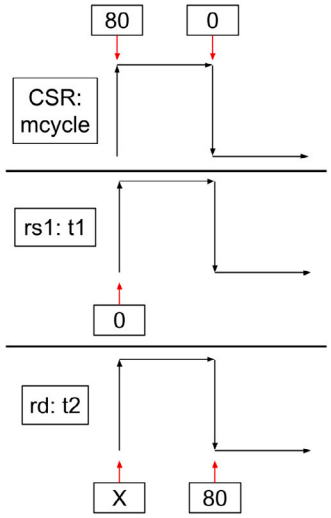


Fig. 5. Example of the read and write concurrent accesses produced between CSRs and GPRs for the atomic modification of the CSRs.

the other two cases, if the write occurs in the same cycle or in the cycle after the count, the event will be lost, as the counter will be overwritten immediately after being counted. This is not an error but is the expected behavior, since if a counter is to be written the intention is to reset the counter, thus losing the previous count. Although all of these scenarios may appear conceptually simple, the hardware synchronization required to implement them is not trivial, due to the fact that performing the atomic accesses in the same cycle can lead to the same signal being driven with multiple inputs, generating combinational loops and other complex problems to debug. Thus, to solve this, the final design adopted the use of shadow registers: this way, after a write on the CSRs, at the falling edge, the value written would be stored in those shadow registers and the value of the CSRs would not be updated in the actual registers until the subsequent rising edge, at which point the increment would also be performed.

3.5. Specific example of operation

Now a complete step-by-step example of how the PMU operates will be presented. In Fig. 6, there is a visual representation of six cycles of execution of the pipeline, one in each row. As can be seen, the pipeline is full, so every cycle a new instruction is written back.

The instruction we will be paying attention to is the one marked in blue. As we can see, during the Instruction Fetch (IF) stage, Fig. 6.a. everything works nominally, and, at this moment, this instruction has triggered the first, third and last events. Respectively, that is because, firstly, a cycle has already been spent by this instruction as it is always the case; secondly, at this point in time the pipeline believes the instruction is going to be retired; and thirdly, the instruction has been fetched; thus the three events triggered.

Meanwhile, to the right, in the count module, we can also see that the events of the instruction which was in the WB stage in the previous cycle are finally counted. One thing to note is that, since after the mentioned instruction was counted the value of the visible counters is 1, we must assume that the previous instruction restarted all the counters of the PMU. In addition, for the rest of this example, it can be considered that all instructions, except the one marked in blue, are nominal arithmetic and logical instructions, and therefore, only generate and count the three events mentioned previously. Furthermore, each event is accrued in its corresponding counter, as shown in Table 1.

Next, in the following cycle, Fig. 6.b., the instruction under analysis has arrived at the decodification stage, where it finds an exception. Since it has been detected at this stage, it could be, for example, an illegal instruction exception. This means that the instruction must not be executed and hence, the retirement event is cleared while the fourth event is set. This is because, in the proposed implementation, as also seen earlier in Table 1, the fourth triggered event is programmed to count exceptions in the *mhpmcounter3* register. Nevertheless, the cycle and fetch events have to be kept set, since the former must always be counted unless inhibited, and the latter because it has already happened and, thus, must be counted anyway.

In the next three cycles, sub Figs. 6.c. – 6.e., the instruction continues through the pipeline as a NOP instruction, chaining the *triggered_events* data structure through the pipeline with the same values set earlier. Finally, when it arrives at the WB stage, Fig. 6.e., due to the exception it does not produce any changes to the state of the CPU. It is also at this moment, once the pipeline has been emptied, that the exception can begin to be managed, and its treatment will begin by fetching its trap handler. Finally, in the next cycle, the instruction abandons the pipeline, pending only the accounting of the events which we can see occur on the subsequent cycle, Fig. 6.f.

This way we can see how, on the one hand, in the previous cycles, sub Figs. 6.c. – 6.e., the cycle, retired instruction and fetch events were counted, since they are regular instructions, as explained earlier. While, on the other hand, for the blue instruction, Fig. 6.f., only cycle and fetch events are incremented due to the exception. And lastly, we can also see a new type of event, an exception, which is also counted at this point.

4. Execution model

Once the PMU was implemented it was time to test that it performed correctly. For this purpose, various pieces of software were characterized and all the events that occurred during its execution were calculated. Then, these programs were executed on the OBC and the results obtained by the PMU were observed to be precisely the ones expected. Table 2 shows an example of these results. In addition, the execution of these programs was followed cycle by cycle, checking that the operation was correct. These pieces of software were based on the quicksort algorithm and modified to create every type of event that the PMU is capable of detecting. More information on why this program was selected and why it is of enough significance for a space-graded on-board computer can be found in [14].

Apropos, when talking about programming languages, an execution model is the way a program is processed so that each of its elements completes its determined function to achieve the final objective of the program. Hence, the two main characteristics of a programming language are its syntax, and its execution model. Then, these execution models, in some cases, can be as simple as executing each line one after the other, meanwhile in other cases, for example in General-Purpose GPU programming (GPGPU) languages, such as with CUDA [43], the use of instruction level parallelism can complicate the model of computation significantly. Another example, with a scope comparable to the use cases elaborated in this paper is [44], where a discussion on an execution model for real-time embedded languages can be found.

Applying the same concept on a lower abstraction layer, analogously, it can be inferred that every CPU has its own execution model. In modern general-purpose CPUs, for instance, they can be so complex that simulating and obtaining a WCET is extremely difficult without big margin errors. This is disfavored in space applications and thus it is common to use simpler CPUs with more deterministic models.

Obtaining the execution model of a CPU can be extremely useful to then be capable of simulating and predicting its behavior, e.g., with the use of cycle-accurate simulators [12]. This results in a reduction of the development time and costs for both software and hardware, and an ease in the complexity of the debugging process, for instance, when adding new functionality to the CPU, to check whether errors or other unintended side effects have been introduced.



Fig. 6. Example of setting and clearing of events by the PMU, and of the counting process.

For these reasons, after having validated the functionality of the PMU with the tests described earlier, all the potential events which altered the CPI of the processor and their corresponding amount of time spent executing were already available, as mentioned on Section 3.3. Therefore, with this information, the current execution model has been calculated, which characterizes the behavior of the presented OBC.

Now this model will be described. For the sake of brevity, as this is not the main topic of the article, but simply a way to provide insight into the advantages gained by integrating the PMU, the explanations of the execution model will not be elaborated in depth, but rather merely an overview of its intricacies will be provided.

For any sequential program, i.e., without instructions that change the control flow, the total number of cycles equals the number of instructions executed, the number of instructions fetched, and the number of hazards found during execution plus 4. These 4 cycles are due to the pipeline being filled at the start of execution and must always be accounted for. Moreover, another thing to consider is that memory access instructions take longer to execute than any other type of instruction, as there is a latency between the petition and receiving the respective data or acknowledgment from memory. In these cases, stores take one extra cycle and loads take two extra cycles to talk to memory, i.e., in total, each of these instructions takes two and three cycles, respectively, to end, with one of them dedicated to the normal propagation to the next stage of the pipeline.

Adding to the complexity, the next case is when a jump or branch instruction is executed. In those cases, 2 cycles must be added to the previous execution model for each of these instructions executed since the CPU commits the jump during the EX-stage and therefore, 2 cycles are lost filling the pipeline again.

Finally, when any trap is encountered (either interrupts or exceptions), they must be considered like a flush and its subsequent refilling of the pipeline. Hence, for both each entry and exit to the trap, 4 cycles are to be accounted for. This is because, on entry, the fetch of the trap handler is not carried out until the pipeline has been completely emptied (as another type of trap could be encountered during the emptying process and therefore take precedence). While, during the MRET execution for the trap handler exit, the pipeline also needs to be emptied to make sure that no instruction previous to it is modifying the *mepc* CSR, to know where to resume the execution, and to make sure all instructions are executed with the appropriate privilege level. There is an exception to this rule when a MRET is immediately followed by another trap, in which case a cycle is lost, and only 7 cycles (instead of 8) are taken for an entire trap.

With all of this information in mind, as mentioned earlier, a program can be characterized with the number of events that would happen during its execution and then checked empirically by hand. Then, with the execution model of the OBC, it can be observed, for example, that the number of cycles counted by the PMU is the same as the number of cycles calculated theoretically to be the length of execution of such a program. Thus, this way, the behavior of the PMU and the OBC can be validated. An example of this can be seen in Table 2.

Table 2
Characterization of a quicksort program with 64 elements. In the first column are all the events that modify the CPI during the execution of the program, as has already been explained, whereas, in the second column are the number of counts of each event, as obtained by the PMU after execution. In the third column, it can be seen the amount of cycles spent per each event due to the OBC execution model, as has been discussed. And finally, in the fourth column are the total cycles employed during the program due to each event. The bottom rightmost cell shows how the theoretical number of cycles obtained from the execution model coincides with the number of cycles measured by the PMU (in the topmost cell of the second column).

Event	Event count	Cycles per event	Total cycles per event
Cycles	119 540	–	–
Retired instructions	32 950	1	32 950
Exceptions	0	8	32 950
External interrupts	0	8	32 950
Timer interrupts	0	8	32 950
Branches	1396	2	35 742
Jumps	1380	2	38 502
Hazards	9519	1	48 021
Loads	13 667	2	75 355
Stores	5675	1	81 030
Fetches	38 506	1	119 536
Initial pipeline filling	–	4	119 540

Table 3
General configuration registers.

Configuration register	Values
mcounteren	0xFFFFFFFF
mcountinhibit	0x000000000 or 0xFFFFFFFF

5. Experimental results

Once the behavior of the PMU has been validated, both with the first tests discussed above as well as with the execution model, as a real-world showcase of its use, the results obtained during the execution of several programs will be exhibited. However, to ensure completeness, for the purpose of this paper, the configuration of the PMU used for the tests will be described first.

The values of the general configuration registers have been what can be seen in Table 3. First, with *mcounteren*, the PMU counters have been set up to also be accessible through user mode to reduce the number of calls to the execution environment and to ease the software development. Secondly, with *mcountinhibit*, the counters were inhibited both during the initial setup and when it was time to read them, to be able to gather the statistics atomically, whereas, the rest of the time, they were uninhibited. Moreover, in regards to the specific configuration registers and which events were programmed to be monitored, they were set up according to the order of Table 1, as explained in the description in Section 3.3.

Throughout the remainder of this section, both the performance metrics and the statistical results obtained by the PMU during the execution of different benchmarks will be exhibited. Furthermore, a comparative analysis between the results of the presented OBC and those of another processor is also conducted. Finally, the resource utilization and power consumption data are analyzed to provide a comprehensive view of the operating efficiency and power consumption details of the processor after the implemented modifications.

5.1. Benchmarks performance results

Before reviewing the results, it should be noted that the performance metrics shown below are with the PMU enabled, since without it, measuring the performance of the OBC would be much more complicated, as this is precisely what we want to achieve with its integration. It could have been possible with an external real-time clock synchronized with the start and end of the execution of each program. However, given the bare-metal nature of the platform, which would make such synchronization rather difficult, and the nanosecond variations that would be encountered at most, it was considered unnecessary since the PMU, by design, does not modify performance. This is because, as described in Section 3.2, the PMU is not intrusive with respect to execution, as it does not affect the way instructions are executed since it only performs its monitoring role, nor does it affect the duration of instructions in the pipeline, since they continue to be written back during the same stage as before the modification. Thus, the results shown are analogous to those without PMU enabled.

Specifically, at the moment of publication, both Dhrystone and CoreMark have been ported to our platform, as they are the most common benchmarks in the industry. Another reason for using them was that RISC-V versions of these benchmarks already existed, such as those in the NEORV32 processor Board Support Packages (BSPs) [45], which significantly eased the porting process. Both benchmarks were compiled with the RISC-V GNU toolchain [46].

Once the porting process was completed, the benchmarks were ready to be executed. Instead of simply providing a single data point with a fixed number of iterations, the benchmarks were left running

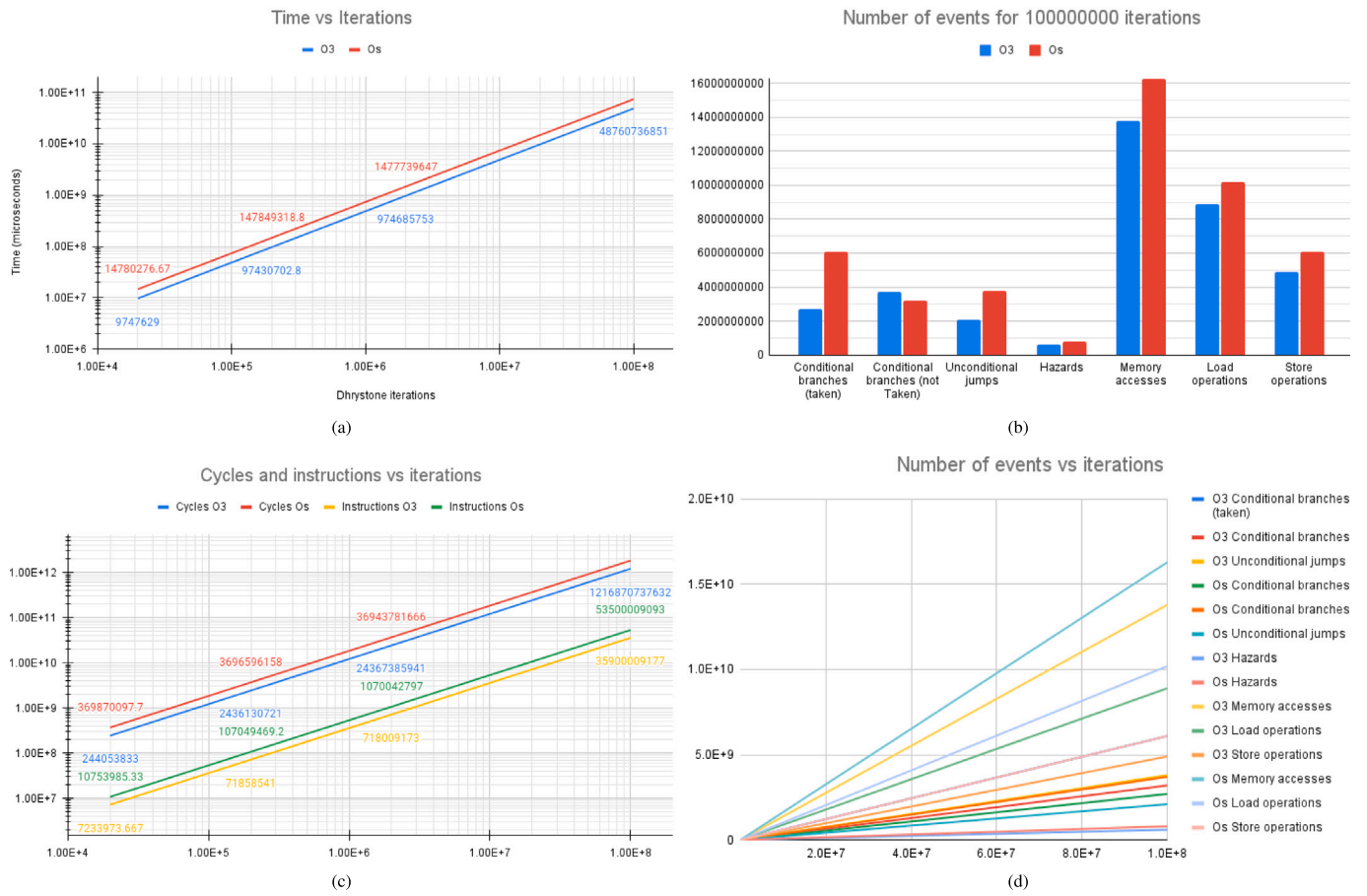


Fig. 7. Results of the Dhrystone benchmark. Figs. 7(a), 7(c) and 7(d) show the linear relationship for each metric over the entire number of iterations. Specifically, the real-time spent executing, cycles and instructions executed and all the other supported events, respectively. It should be noted that the axes are in logarithmic scale. Meanwhile, Fig. 7(b) exhibits the count of each event that occurred on tests with 1000000000 iterations.

for different lengths of time and with different compiler optimizations to examine the consistency of the results. This approach enabled not only performance evaluation but also to test the temporal consistency of the processor, and thus validate the processor's deterministic and linear behavior. It is important to note that, when tests with the same number of iterations were conducted, the results were identical across all experiments. These tests were successfully conducted on various computers with multiple evaluation boards to verify the reproducibility of the results. The test environments consisted of the Vivado HDL Suite version 2018.3 [47] used to program two different Nexys4-DDR evaluation boards [48]. The summary of all executions can be seen in Figs. 7 and 8 respectively for each benchmark. Meanwhile, the performance results for each number of iterations tested are in Tables 4 and 5.

As can be seen in these results, the execution time, along with the number of cycles and instructions executed, and every other type of supported event, increases linearly with the amount of iterations of each benchmark, fulfilling the established validation objective. This is due to the fact that, as explained in Section 3.2, these tests were run without cache. Moreover, it is noteworthy that identical results were obtained across multiple tests for each iteration count, reinforcing the robustness of the findings.

Also, in Tables 4 and 5, the CPI and other performance metrics of both Dhrystone and CoreMark can be observed. In order to understand these results, it must be mentioned again that performance has not been a priority during the development of this OBC; instead, the objective has always been to create a test bed to work on different proofs of concept of novel tools to increase the reliability of the hardware and software developed, as discussed in previous sections. Nonetheless, some performance improvements are already planned for the future.

In addition, the results are very sensitive to the optimization level used during the compilation process, as is common with these kinds of benchmarks. For example, the execution time is significantly lower on the tests compiled with the "-O3" flag instead of optimizing for program size "-Os". Moreover, the distribution of the remaining events shifts remarkably depending on the optimization flags, as can be seen in Figs. 7(b) and 8(b). For instance, as also seen in Figs. 7(c) and 8(c), the number of instructions, especially jumps (both conditional and unconditional), and memory access operations, are significantly lower in the versions compiled with the "-O3" flag, demonstrating the great predictive and optimization capabilities of existing compilers. This is especially notable in the Dhrystone benchmark, with taken and unconditional jumps reducing nearly in half, whereas in CoreMark, although these differences still exist, they are much more succinct. Of special significance is the *-fpredictive-commoning* optimization, which reuses computations made during the flow of a program, most notably memory loads and stores performed in previous iterations of loops, thus the variations observed in the figures. These disparities explain most of the temporal deviation between execution times.

To try and mitigate the discrepancies created with software optimizations, the addition of newer and improved benchmarks such as Embench [49] is a work in progress for the foreseeable future. Furthermore, a port to our RISC-V platform of the Boot Software of the Instrument Control Unit (ICU) of the Energetic Particle Detector (EPD) currently on-board the Solar Orbiter spacecraft is already ongoing at the moment of publication.

Finally, it is important to note that the PMU has already proven extremely useful during our workflow, as it has improved the development of both our hardware and our software, facilitating the

Table 4
Performance results of the Dhrystone benchmark.

Optimization	Os	Os	Os	Os	O3	O3	O3	O3
Iterations	100000000	2 000 000	200 000	20 000	100000000	2 000 000	200 000	20 000
Real Time (µs)	73852000010	1477739647	147849319	14 780 277	48760736851	974685753	97 430 703	9747629
Cycles	1846300363478	36943781666	3696596158	369870098	1216870737632	24367385941	2436130721	244053833
Instructions	53500009093	1070042797	107049469	10 753 985	35900009177	718009173	71 858 541	7233974
CPI	34.5103	34.5255	34.5317	34.3938	33.8961	33.9374	33.9018	33.7372
Exceptions	0	0	0	0	0	0	0	0
External interrupts:	0	0	0	0	0	0	0	0
Timer interrupts	0	0	0	0	0	0	0	0
Conditional branches (taken)	6100002926	122002925	12 202 923	1 222 924	2700002952	54002950.67	5 402 949	542 948
Conditional branches (not Taken)	3200000038	64 000 038	6 400 038	640 038	3700000063	74 000 063	7 400 063	740 063
Unconditional jumps	3800000093	76 000 093	7 600 093	760 093	2100000027	42 000 027	4 200 027	420 027
Hazards	800002913	16 002 912	1 602 910	162 910	600002955.5	12002954.67	1 202 953	122 952
Memory accesses	16300003000	326002999	32 602 997	3 262 997	13800003064	276003063.5	27 603 061	2763059.5
Load operations	10200002935	204002934	20 402 932	2 042 931	8900002999	178002997.7	17 802 996	1782995
Store operations	6100000065	122000065	12 200 065	1 220 064	4900000065	98 000 065	9 800 065	980 065
Dhrystones ($DMIPS/s$)	1354.0595	1353.4192	1352.7307	1353.1558	2050.8358	2051.9482	2052.7444	2051.7855
VAX $DMIPS/s$	0.7707	0.7703	0.7699	0.7702	1.1672	1.1679	1.1683	1.1678
µs per run of Dhrystone	738.5200	738.8698	739.2466	739.0138	487.6074	487.3429	487.1535	487.3815

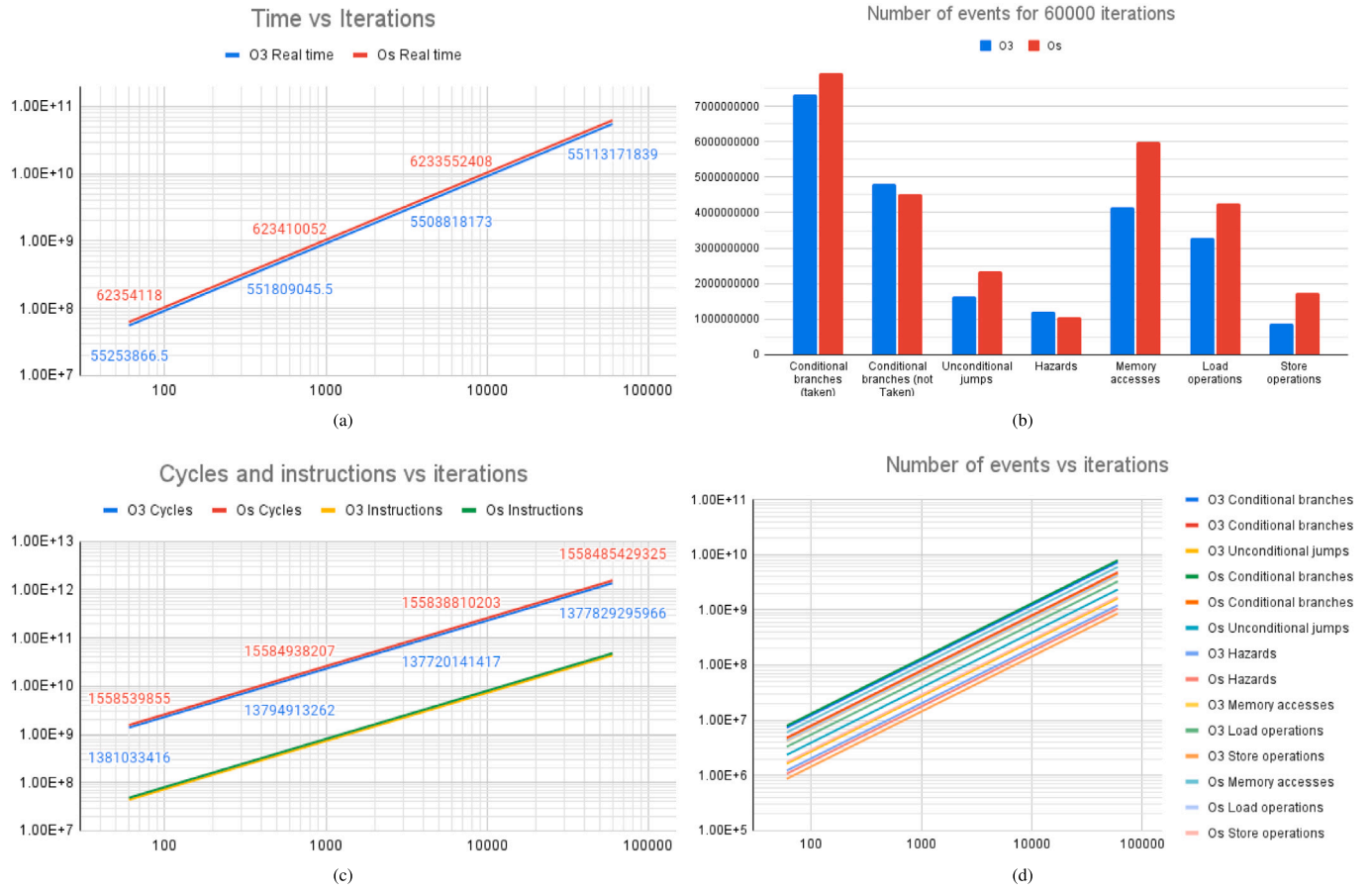


Fig. 8. Results of the CoreMark benchmark. Figs. 8(a), 8(c) and 8(d) show the linear relationship for each metric over the entire number of iterations. Specifically, the real-time spent executing, cycles and instructions executed and all the other supported events, respectively. It should be noted that the axes are in logarithmic scale. Meanwhile, Fig. 8(b) exhibits the count of each event that occurred on tests with 60,000 iterations.

debugging of issues during the OBC design and implementation and adding reliability to our tests.

5.2. Comparison with other processors

In this subsection, a comparative analysis is conducted between the statistical results obtained by executing two different sets of tests on the presented OBC and an alternative processor. This is because, although with the results showcased in Section 4 the behavior of the proposed CPU and the PMU had already been internally validated, its comparison with another processor would help validate them against an external reference. Moreover, such comparison would help further illustrate the impact achieved with the modifications proposed in the design of this PMU.

The processor selected for the comparison has been the NEORV32 [45]. Its choice is informed by its prominence within the RISC-V community, its alignment with our area of focus, as even though it is not specifically targeted for space applications, it is also designed as a microcontroller, and our existing familiarity with its architecture, as discussed in the preceding subsection. Additionally, despite challenges stemming from differences in event coverage arising from distinct pipeline structures, as its architecture is not fully pipelined, key metrics such as the number of instructions executed, jumps, and memory operations still enable meaningful comparison.

Table 5
Performance results of the Coremark benchmark.

Optimization	Os	Os	Os	Os	O3	O3	O3	O3
Iterations	60 000	6000	600	60	60 000	6000	600	60
Real Time (µs)	62339417173	6233552408	623410052	62 354 118	55113171839	5508818173	551809045.5	55253866.5
Cycles	1558485429325	155838810203	15584938207	1558539855	1377829295966	137720141417	13794913262	1381033416
Instructions	48134147340	4813416078	481341900	48 135 534	43721970101	4372198097	437220062	43723094
CPI	32.37795859	32.37592754	32.3781047	32.37816125	31.51343118	31.49906257	31.55141875	31.58590322
Exceptions	0	0	0	0	0	0	0	0
External interrupts:	0	0	0	0	0	0	0	0
Timer interrupts	0	0	0	0	0	0	0	0
Conditional branches (taken)	7931315815	793131617	79 313 245	7 931 360	7331892933	733189424	73 319 043	7332035
Conditional branches (not Taken)	4529380830	452938073	45 293 730	4 529 363	4813800687	481379947	48 137 876	4813666
Unconditional jumps	2348400587	234840174	23 484 047	2 348 520	1627148804	162715021	16 271 562	1627297
Hazards	1062969079	106296958	10 629 769	1 063 027	1216686573	121668690	12 166 892	1216722
Memory accesses	5999117629	599912373	59 991 349	5 999 745	4157900927	415790466	41 579 202	4158294
Load operations	4264161951	426416559	42 641 751	4 264 539	3294413599	329441606	32 944 278	3294674
Store operations	1734955678	173495814	17 349 598	1 735 206	863487328	86 348 860	8 634 924	863 620
Coremarks	0.9625	0.9625	0.9624	0.9622	1.0887	1.0892	1.0873	1.0859

The first set of tests can be found in Table 6. In it, the results of executing the Coremark benchmark with the same configuration as in the previous subsection on both processors can be found. In the interest of clarity and conciseness, these analyses focus solely on the outcomes derived from the Coremark benchmark with the -Os compiler optimization flag. While the preceding section encompassed tests for both Coremark and Dhrystone benchmarks, including results from both -Os and -O3 compiler optimization flags, ultimately, the same findings were reached for all tests. Consequently, it has been concluded that the results obtained with the Coremark benchmark under the -Os optimization flag adequately represent the performance characteristics of the processors under study. Thus, to avoid redundancy and maintain relevance, the decision has been made not to duplicate the analysis by including additional result sets in this comparison.

It is important to acknowledge there are some significant disparities between the results presented in this section and those in the preceding one. These differences arise from the adjustments made to the memory interface of the proposed OBC to ensure closer comparability with the NEORV32. Unlike the NEORV32, which operates solely with integrated memory, the results shown in Section 5.1 utilize the external memory provided by the Nexys4-DDR development board [48]. This approach was chosen for its broader applicability, reproducibility, and readiness for future expansions. However, to ease the comprehension of the comparison with the NEORV32, modifications were made to the memory access mechanism. Specifically, rather than utilizing the Memory Interface Generator (MIG) controller for external memory access, the system's main memory was synthesized within the FPGA. This alteration significantly reduces latency in accesses to memory, leading to improved performance as observed in the respective columns for the presented OBC in Table 6. Nevertheless, it can be seen that despite these performance enhancements, the remaining statistical results remain unchanged.

Upon examination of Table 6, it is evident that the statistics obtained with the PMU of each processor are consistent with each other. Firstly, the number of instructions precisely matches for each individual set of tests, considering even the executions featuring vastly different numbers of iterations. Secondly, while the NEORV32 lacks events to differentiate between each type of branch instruction, the results for taken and total branch instructions align precisely with the totals obtained in the proposed PMU. Lastly, even though the number of cycles and the real-time duration of execution are inherently non-comparable, due to differences in the microarchitectures, their results remain coherent. The proposed processor, featuring a pipelined architecture, exhibits a lower CPI compared to the NEORV32's multicycle architecture with pipelined fetch and execute stages, resulting in slightly longer execution times for the latter in each test. The congruence displayed across these tests underscores the reliability and accuracy of the measurements from the proposed PMU.

Furthermore, to better illustrate the differences in processor count influenced by the proposed PMU design in more intricate scenarios, we introduce Table 7. This table offers a comparative analysis wherein an exception is encountered every 100000 instructions. The objective of this set of tests is to scrutinize the impact of the proposed PMU design on its statistical outcomes in more real-world scenarios where traps are commonplace, rather than only through benchmarks with nominal runs. These tests were also configured to execute for different amounts of time, to evaluate the temporal consistency and reproducibility of the results.

In contrast to the previous set of tests, this analysis poses greater complexity, requiring an understanding of how each processor establishes its counting strategies. For instance, in the NEORV32, all control flow transfer operations are collectively tallied within a single counter, labeled 'Total jumps taken' in Table 7. This counter includes all branch instructions, both conditional and unconditional, as well as trap entries and exits. However, in the event named 'Total jumps instructions', only the jumps due to instructions are accounted for, both taken and not taken. In contraposition, the proposed design distinguishes each type of branch instruction and assumes the occurrence of an entry and exit jump for every trap, as they must necessarily always be realized. Therefore, the NEORV32 appears to register two additional branches for each trap raised within the former counter, a divergence absent in the latter counter. Thus, these discrepancies emerge from counting the entry and exit jumps for each trap within that same first counter, whereas the latter counter does indeed track the same events, resulting in identical counts. Consequently, the counts for these events are equivalent, with the disparities arising solely from the counting strategy of each processor. Meanwhile, the counts for the remaining events tracked in both processors match exactly.

Nevertheless, it is evident that the NEORV32 processor registers a greater total count of instructions executed compared to the proposed design, with the increment directly correlating to the number of encountered traps. This disparity underscores the limitation discussed throughout this article, which serves as the motivation for the proposed design. Unlike the NEORV32, which lacks the capability to retract an event once triggered, the proposed design aims to address this issue, ensuring a more reliable count.

Lastly, it is important to note that the example portrayed here represents a relatively mild scenario in comparison with the severe examples mentioned in Section 3.2 when the motivation of this design was presented. This is because, as briefly mentioned earlier, the NEORV32 possesses a multicycle architecture and thereby only a single instruction is executed at a time. The rationale for this design is also to be able to provide precise trap control [50]. Therefore, as there is only one instruction executing, when a trap arrives, it is easier to control which events need to be reverted in comparison with a pipelined architecture, where multiple instructions are executing concurrently. Hence, it is reasonable to infer that processors with less emphasis on performing precise and accurate counting in trap situations, may be even more significantly impacted in this scenario.

Table 6
Comparison between the results from executing the Coremark benchmark on each CPU for each number of iterations.

Processor	RV32Xtrace	NEORV32	RV32Xtrace	NEORV32	RV32Xtrace	NEORV32	RV32Xtrace	NEORV32
Optimization	Os	Os	Os	Os	Os	Os	Os	Os
Iterations	60 000	60 000	6000	6000	600	600	60	60
Real Time (µs)	5777969880	7939301740	577797146	793930514	57 779 763	79 393 149	5 778 134	7939655
Cycles	144449246994	198482543494	14444928651	19848262843	1444494086	1984828714	144453360	198491365
Instructions	48134147340	48134147340	4813416078	4813416078	481341900	481341900	48 135 534	48135534
CPI	3,0009723	4,1235288	3,0009723	4,1235294	3,0009731	4,1235319	3,0009713	4,1235933
Exceptions	0	0	0	0	0	0	0	0
External interrupts:	0	0	0	0	0	0	0	0
Timer interrupts	0	0	0	0	0	0	0	0
Conditional branches (taken)	7931315815	n.a.	793131617	n.a.	79 313 245	n.a.	7 931 360	n.a.
Unconditional jumps	2348400587	n.a.	234840174	n.a.	23 484 047	n.a.	2 348 520	n.a.
Total jumps taken	10279716402	10279716402	1027971791	1027971791	102797292	102797292	10 279 880	10279880
Conditional branches (not Taken)	4529380830	n.a.	452938073	n.a.	45 293 730	n.a.	4 529 363	n.a.
Total jumps	14809097232	14809097232	1480909864	1480909864	148091022	148091022	14 809 243	14809243
Hazards	1062969079	n.a.	106296958	n.a.	10 629 769	n.a.	1 063 027	n.a.
Memory accesses	5999117629	n.a.	599912373	n.a.	59 991 349	n.a.	5 999 745	n.a.
Load operations	4264161951	4264161951	426416559	426416559	42 641 751	42 641 751	4 264 539	4264539
Store operations	1734955678	1734955678	173495814	173495814	17 349 598	17 349 598	1 735 206	1735206
Coremarks	10,384270123	7,557339670	10,384267284	7,557336434	10,384258585	7,557327144	10,38397517	7,557003421

Table 7
Comparison between the statistical results on each processor after the execution of the different tests with exceptions.

Processor	RV32Xtrace	NEORV32	RV32Xtrace	NEORV32	RV32Xtrace	NEORV32
Optimization	Os	Os	Os	Os	Os	Os
Iterations	1	1	10 000 000	10 000 000	1000000000	1000000000
Real Time (µs)	12	16	100208622	131524896	17445198786	23229902382
Cycles	305	404	2505215566	3288122409	436129969670	580747559552
Instructions	91	92	801604662	801604762	139815669906	139815679906
CPI	3,329670	4,391304	3,1252507	4,1019247	3,119321	4,153665
Exceptions	1	1	100	100	10 000	10 000
External interrupts:	0	0	0	0	0	0
Timer interrupts	0	0	0	0	0	0
Conditional branches (taken)	4	n.a.	160499981	n.a.	28329532664	n.a.
Unconditional jumps	8	n.a.	50 000 102	n.a.	9294977298	n.a.
Total jumps taken	12	14	210500083	210500283	37624509962	37624529962
Conditional branches (not Taken)	7	n.a.	148700093	n.a.	24822309986	n.a.
Total jump instructions	19	19	359200176	359200176	62446819948	62446819948
Hazards	14	n.a.	20 000 705	n.a.	2000070005	n.a.
Memory accesses	46	n.a.	40 003 605	n.a.	4000360005	n.a.
Load operations	25	25	30 001 804	30 001 804	3000180004	3000180004
Store operations	20	20	10 001 801	10 001 801	1000180001	1000180001

5.3. Utilization and power results

Once the performance results have been described, now the resource and power utilization will be portrayed. These results are particularly important in this case because of the field of application of this on-board computer, due to the significant power and mass limitations in the space industry, as it is well known.

With regard to resource utilization, Tables 8 and 9 present a summary of the main resources used by the OBC, both with and without the PMU enabled, respectively. All of these metrics were extracted from the Vivado Suite analysis report tool and the results provided are those obtained during synthesis with exactly the same settings selected for both projects.

As can be imagined, most of the extra resources used by the design with PMU are in the CSR unit, due to the new registers and logic implemented; being the remaining extra resources allocated in the inter-stage registers where the events are monitored and then propagated through the remaining pipeline, with the *triggered_events* data structure.

Table 10 shows the relative difference between each design and the total resources of the FPGA used during the evaluation process [48]. Here, it shows an increase of 2.65% in LUT logic resources and 0.85% in Slice Registers used with the PMU enabled. Though a notable increment, it is to be expected due to the significant amount of logical resources required to control the accountability of the events. This is because, in comparison with the GPR registers, here the VHDL optimizer does not use as many F7 multiplexers resources to access the CSR registers, due to the higher complexity for accessing them, having to use LUT logic instead. On the other hand, the increment in Slice Registers is significantly smaller and justified by the higher number of registers to accommodate the PMU counters.

Table 8
Resources utilization of the OBC with the PMU enabled.

Component	Slice LUTs	Slice Registers	F7 Muxes	Block RAM
RV32	6498	5347	292	2
clint	77	72	0	0
decode	334	44	0	0
execute	634	0	1	0
fu	74	0	0	0
if	22	35	0	0
mc	205	284	0	0
memory	120	72	0	0
pipeline_logic	406	203	0	0
plic	84	1	0	0
prom	2	0	0	2
reg	3853	3398	291	0
CSR	3181	2406	35	0
GPR	640	992	256	0
rexmem	77	172	0	0
ridex	137	296	0	0
rifid	80	405	0	0
rmemwb	87	166	0	0
sel_plic_clint	50	99	0	0
timer	245	134	0	0

Next, in regard to the power usage of the implementation, two methods were used to obtain the power requirements measurements. First, the metrics provided by the simulations from the Vivado power

Table 9
Resources utilization of the OBC with the PMU disabled.

Component	Slice LUTs	Slice Registers	F7 Muxes	Block RAM
RV32	4817	4246	259	2
clint	76	72	0	0
decode	327	44	0	0
execute	634	0	1	0
fu	74	0	0	0
if	22	35	0	0
mc	174	280	0	0
memory	118	72	0	0
pipeline_logic	381	203	0	0
plic	88	1	0	0
prom	2	0	0	2
reg	2257	2342	258	0
CSR	1585	1350	2	0
GPR	640	992	256	0
rexmem	67	159	0	0
ridex	129	283	0	0
rifid	82	104	0	0
rmemwb	80	152	0	0
sel_plic_clint	50	99	0	0
timer	245	134	0	0

Table 10
Relative resource utilization in relation to the maximum resources available of the FPGA board.

	Slice LUTs	Slice Registers	F7 Muxes	Block RAM
Total resources board	63 400	126 800	31 700	135
RV32Xtrace PMU	6498	5347	292	2
% RV32Xtrace PMU	10.25%	4.22%	0.92%	1.48%
RV32Xtrace NO PMU	4817	4246	259	2
% RV32Xtrace NO PMU	7.60%	3.35%	0.82%	1.48%

Table 11
Power measurements of the OBC with the PMU enabled.

Test	Voltage (V)	Current (A)	Power (W)
Baseline	5	0.225	1.125
Programming	5	0.250	1.250
Bootloader	5	0.375	1.875
Uploading	5	0.376	1.880
Dhrystone	5	0.426	2.130
Coremark	5	0.426	2.130

Table 12
Power measurements of the OBC with the PMU disabled.

Test	Voltage (V)	Current (A)	Power (W)
Baseline	5	0.225	1.125
Programming	5	0.260	1.300
Bootloader	5	0.373	1.865
Uploading	5	0.375	1.875
Dhrystone	5	0.425	2.125
Coremark	5	0.425	2.125

analysis tool were extracted. These reflect the power consumption after routing and placement for the selected FPGA chip, i.e., that of the Nexys 4-DDR board [48]. The estimates obtained indicated that the OBC implementation with PMU consumed 110 mW, whereas without PMU it employed 104 mW. Thus, the difference in consumption with the Artix XC7A100T-1CSG324 FPGA was 6 mW more for the implementation with PMU. Secondly, to verify that these estimates were correct, the FPGA evaluation boards were programmed with two different IPcores, one containing the RV32Xtrace OBC with the presented PMU enabled and another with the PMU disabled. This board was then connected to a configurable power supply through which the voltage and current measurements could be observed. After this, the power usage was calculated.

In Tables 11 and 12, the results of these tests can be found. For each IPcore configuration, the following set of tests were conducted. First, the baseline current was measured with the board connected to power but without any IPcore programmed into its memory. Secondly, the board was flashed with a bootloader developed in-house based on the NEORV32 BSP [45]. Hence, the average measurements were obtained both during the programming process and afterward when the bootloader was already running in a safe state. Lastly, the bootloader was used to upload to the board the benchmarks mentioned in Section 5.1 through its serial connection. Here, the metrics of the average power employed during each of the benchmarks, were also collected, as can again be seen in the Tables 11 and 12.

As shown in these tables, the power requirements needed by the OBC after the development of the proposed PMU have obviously increased, although by a minimal margin. One aspect worth noting is that, in order to obtain these results, the measured current and power values were averaged, as they fluctuated depending on board temperature and other confounding variables, which explains the anomaly on the *Programming* row, where the power is lower on the IPcore with PMU. Nevertheless, the rest of the measurements are consistent with the difference being negligible, with the maximum increase under all conditions due to the integration of the PMU of only 10 mW. As can be seen, these metrics overlap with the estimates from Vivado since, although the former measure the consumption of the entire board and the latter only that of the FPGA, the relative difference is comparable.

6. Conclusions and future work

Computing critical systems and, in particular, space-graded systems within the scope of this article, must adhere to the strictest timing requirements to ensure its correct operation and the safety of each of their components. To ease the development of this type of systems, one tool which is usually required is a statistical unit with which to extract timing and other useful information about the behavior of the system. Common performance monitoring units present the issue that it is usually very hard to match the instructions with the events that occurred during their execution on the CPU. To solve that problem, this paper displays the detailed design of a PMU that synchronizes the event counting with the instruction execution.

The design was based on an existing pipelined RISC-V processor to which the statistical counters were added. The modification adds logic to each inter-stage register to check whether the programmed events have occurred, storing this information on a new data structure, which is fed from one register to the next. This way, the event increment only occurs after the corresponding instruction has finally been written back to the GPRs and, therefore, the match between the event and the executed instruction can be stored unequivocally.

In order to validate the design, the events triggered by several programs were calculated, and then they were executed on the on-board computer. The results obtained matched perfectly with the planned outcome, demonstrating the correct operation of the presented PMU. Moreover, the events were specially selected to help reconstruct the execution model of the OBC, as discussed in more detail in this manuscript. Thus, the ability to support new events during development was tested, demonstrating the fast and easy extensibility of the PMU with minimal development time and costs. Likewise, the CoreMark and Dhrystone benchmarks were also ported to the proposed platform and compared with the results from an external reference, obtaining the results analyzed above. This characterization of the execution model and the results presented allow us to confirm the correct behavior of the HPM proposed for this article.

Furthermore, a few ideas already exist on how to enhance the current design. One area of improvement is the creation of a new way to extract these synchronized event data, for example, via an upgraded trace mechanism. Also, several mechanisms have been proposed to improve processor performance. Additionally, to refine the data on the behavior and performance of the OBC, further benchmarks and the boot software from the EPD instrument of the Solar Orbiter mission will be ported to the platform.

Funding

This work has been supported by two predoctoral aids: "Design and Implementation of Leon Processor Enhancements on FPGA" of the Youth Employment Initiative (YEI) of the European Social Fund (ESF) and the Spanish Ministerio de Ciencia, Innovación y Universidades (MCIN), under the Operational Program of Youth Employment (POEJ), grant PEJ2018-004178-A; and the contract: PRE2020-094740 under the project "Energetic Particle Detector en Solar Orbiter: fase E, calibración y explotación de datos" reference: PID2019-104863RB-I00, funded by the MCIN, Spain (DOI: 10.13039/501100011033), the Spanish Agencia Estatal de Investigación (AEI) and by the ESF+.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] C. Redmond, RISC-V: The Open Era of Computing, Tokio, Japan, 2021, URL https://web.archive.org/web/20230119111955/https:/riscv.org/wp-content/uploads/2021/05/RISC-V-New-Era-04-19-2021.pptx.

[2] JPL, High-Performance Space Computing Technology, JPL, NASA, 2020, NASA SBIR 2021 Phase I Solicitation, URL https://web.archive.org/web/20220814221207/https://sbir.nasa.gov/printpdf/68381.

[3] S. Di Mascio, A. Menicucci, E. Gill, G. Furano, C. Monteleone, Leveraging the openness and modularity of RISC-V in space, J. Aerosp. Inf. Syst. 16 (11) (2019) 454–472, http://dx.doi.org/10.2514/1.I010735, Publisher: American Institute of Aeronautics and Astronautics. URL https://arc.aiaa.org/doi/10.2514/1.I010735.

[4] N.-J. Wessman, F. Malatesta, J. Andersson, P. Gomez, M. Masmano, V. Nicolau, J.L. Rhun, G. Cabo, F. Bas, R. Lorenzo, O. Sala, D. Trilla, J. Abella, De-RISC: the first RISC-V space-grade platform for safety-critical systems, in: 2021 IEEE Space Computing Conference, SCC, 2021, pp. 17–26, http://dx.doi.org/10.1109/SCC49971.2021.00010.

[5] N.-J. Wessman, F. Malatesta, S. Ribes, J. Andersson, A. García-Vilanova, M. Masmano, V. Nicolau, P. Gomez, J.L. Rhun, S. Alcaide, G. Cabo, F. Bas, P. Benedicte, F. Mazzocchetti, J. Abella, De-RISC: A complete RISC-V based space-grade platform, in: 2022 Design, Automation & Test in Europe Conference & Exhibition, DATE, (ISSN: 1558-1101) 2022, pp. 802–807, http://dx.doi.org/10.23919/DATE54114.2022.9774557.

[6] D. Meziat, J. Sequeiros, J. Medina, S. Sánchez, CDPU for SOHO-CEPAC collaboration, Microprocess. Microprogr. 37 (1) (1993) 41–44, http://dx.doi.org/10.1016/0165-6074(93)90012-A, URL https://www.sciencedirect.com/science/article/pii/016560749390012A.

[7] S. Sanchez, D. Meziat, M. Carbajo, J. Medina, E. Bronchalo, J. Rodriguez-Pacheco, L. del Peral, Control system for a low energy particle detector, in: Proceedings. 24th EUROMICRO Conference (Cat. No.98EX204), Vol. 1, 1998, pp. 216–220, http://dx.doi.org/10.1109/EURMIC.1998.711803.

[8] O.R. Polo, P. Parra, M. Knobluch, I. Garcia, J. Fernandez, S. Sanchez, M. Angulo, Component based engineering and multi-platform deployment for nanosatellite on-board software, in: DASIA 2012 - DAta Systems in Aerospace, in: ESA Special Publication, vol. 701, 2012, p. 34, URL https://ui.adsabs.harvard.edu/abs/2012ESASP.701E..34P.

[9] S. Sánchez, M. Prieto, Ó.R. Polo, P. Parra, A.d. Silva, Ó. Gutiérrez, R. Castillo, J. Fernández, J. Rodríguez-Pacheco, HW/SW co-design of the instrument control unit for the energetic particle detector on-board solar orbiter, Adv. Space Res. 52 (6) (2013) 989–1007, http://dx.doi.org/10.1016/j.asr.2013.05.029, URL https://www.sciencedirect.com/science/article/pii/S0273117713003360.

[10] A. da Silva, S. Sánchez, Ó.R. Polo, P. Parra, Injecting faults to succeed. verification of the boot software on-board solar orbiter's energetic particle detector, Acta Astronaut. 95 (2014) 198–209, http://dx.doi.org/10.1016/j.actaastro.2013.11.004, URL https://www.sciencedirect.com/science/article/pii/S0094576513003962.

[11] Ó. R. Polo, J. Sánchez, A. da Silva, P. Parra, A. Martínez Hellín, A. Carrasco, S. Sánchez, Reliability-oriented design of on-board satellite boot software against single event effects, J. Syst. Archit. 114 (C) (2021) http://dx.doi.org/10.1016/j.sysarc.2020.101920.

[12] J. Sánchez, A. da Silva, P. Parra, Ó. R. Polo, A. Martínez Hellín, S. Sánchez, ARINC653 channel robustness verification using LeonViP-MC, a LEON4 multicore virtual platform, Electronics 10 (10) (2021) 1179, http://dx.doi.org/10.3390/electronics10101179, Number: 10 Publisher: Multidisciplinary Digital Publishing Institute. URL https://www.mdpi.com/2079-9292/10/10/1179.

[13] B. Losa, P. Parra, A.D. Silva, Ó.R. Polo, J.I.G. Tejedor, A. Martínez, J. Sánchez, S. Sánchez, D. Guzmán, Memory management unit for hardware-assisted dynamic relocation in on-board satellite systems, IEEE Trans. Aerosp. Electron. Syst. (2023) 1–17, http://dx.doi.org/10.1109/TAES.2023.3284419, Conference Name: IEEE Transactions on Aerospace and Electronic Systems.

[14] I. Gamino del Río, A. Martínez Hellín, Ó. R. Polo, M. Jiménez Arribas, P. Parra, A. da Silva, J. Sánchez, S. Sánchez, A RISC-V processor design for transparent tracing, Electronics 9 (11) (2020) 1873, http://dx.doi.org/10.3390/electronics9111873, URL https://www.mdpi.com/2079-9292/9/11/1873.

[15] A. Waterman, K. Asanovic, The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA Document, Tech. Rep., RISC-V International, 2019, URL https://web.archive.org/web/20230126153534/https://github.com/riscv/riscv-isa-manual/releases/download/Ratified-IMAFDQC/riscv-spec-20191213.pdf.

[16] Gaisler, Product Brief: NOEL-V: Highly Configurable RISC-V Processor, Tech. Rep., Cobham Gaisler, Goteburg, Sweden, 2022, URL https://web.archive.org/web/20221023185050/https://gaisler.com/products/noel-v/Product_Brief_NOEL-V_August2022.pdf.

[17] EIC, De-RISC: Dependable Real-time Infrastructure for Safety-Critical Computer, European Innovation Council (EIC), European Commission, 2022, H2020, URL https://doi.org/10.3030/869945.

[18] EPI, Press Release: Successful Conclusion of European Processor Initiative Phase One, European Processor Innitiave (EPI), 2021, H2020, URL https://www.european-processor-initiative.eu/dissemination-material/press-release-successful-conclusion-of-european-processor-initiative-phase-one/.

[19] A. Waterman, K. Asanovic, J. Hauser, The RISC-V Instruction Set Manual, Volume Ii: Privileged Architecture, Document Version 20211203, Tech. Rep., RISC-V International, 2021, URL https://web.archive.org/web/20220719154745/https://github.com/riscv/riscv-isa-manual/releases/download/Priv-v1.12/riscv-privileged-20211203.pdf.

[20] T. M. Johnson, Intel Nehalem Performance Monitoring Unit Programming Guide, Tech. Rep., Intel, 2010, URL https://web.archive.org/web/20230529112624/https://www.intel.com/content/dam/develop/external/us/en/documents/30320-nehalem-pmu-programming-guide-core.pdf.

[21] ARM, Cortex-A5 Technical Reference Manual r0p1, Tech. Rep., ARM, 2009, pp. 198–219, URL https://web.archive.org/web/20230529113300/https://documentation-service.arm.com/static/602f9ee883844146ae0444d8?token=.

[22] V. Salapura, K. Ganesan, A. Gara, M. Gschwind, J.C. Sexton, R.E. Walkup, Next-generation performance counters: Towards monitoring over thousand concurrent events, in: Proceedings of the ISPASS 2008 - IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS '08, IEEE Computer Society, USA, 2008, pp. 139–146, http://dx.doi.org/10.1109/ISPASS.2008.4510746.

[23] J. Gaisler, E. Catovic, M. Isomaki, K. Glembo, S. Habinc, GRLIB IP Core User's Manual, Tech. Rep., Cobham Gaisler, 2023, pp. 1479–1486, URL https://web.archive.org/web/20240204105018if_/https://www.gaisler.com/products/grlib/grip.pdf#G90.966345.

[24] N. Ho, P. Kaufmann, M. Platzner, A hardware/software infrastructure for performance monitoring on LEON3 multicore platforms, in: 2014 24th International Conference on Field Programmable Logic and Applications, FPL, (ISSN: 1946-1488) 2014, pp. 1–4, http://dx.doi.org/10.1109/FPL.2014.6927437.

[25] B. Sprunt, The basics of performance-monitoring hardware, IEEE Micro 22 (4) (2002) 64–71, http://dx.doi.org/10.1109/MM.2002.1028477, Conference Name: IEEE Micro.

[26] J. Dean, J. Hicks, C. Waldspurger, W. Weihl, G. Chrysos, ProfileMe: hardware support for instruction-level profiling on out-of-order processors, in: Proceedings of 30th Annual International Symposium on Microarchitecture, (ISSN: 1072-4451) 1997, pp. 292–302, http://dx.doi.org/10.1109/MICRO.1997.645821.

[27] J. Domingos, P. Tomás, L. Sousa, Supporting RISC-v performance counters through performance analysis tools for linux (perf), 2021, http://dx.doi.org/10.48550/arXiv.2112.11767, CoRR abs/2112.11767. URL https://arxiv.org/abs/2112.11767.

[28] T. Scheipel, F. Mauroner, M. Baunach, System-aware performance monitoring unit for RISC-V architectures, in: 2017 Euromicro Conference on Digital System Design, DSD, 2017, pp. 86–93, http://dx.doi.org/10.1109/DSD.2017.28.

[29] M. Lei, T.-Y. Yin, Y.-C. Zhou, J. Han, Highly reconfigurable performance monitoring unit on RISC-V, in: 2020 IEEE 15th International Conference on Solid-State & Integrated Circuit Technology, ICSICT, 2020, pp. 1–3, http://dx.doi.org/10.1109/ICSICT49897.2020.9278263.

[30] RISC-V, RISC-V Count Overflow and Mode-Based Filtering Extension: Sscofpmf, Tech. Rep., RISC-V International, 2021, URL https://web.archive.org/web/20230304085315/https://github.com/riscv/riscv-count-overflow/releases/download/v0.5.2/Sscofpmf.pdf.

[31] J. Rodríguez-Pacheco, R.F. Wimmer-Schweingruber, G.M. Mason, G.C. Ho, S. Sánchez-Prieto, M. Prieto, C. Martín, H. Seifert, G.B. Andrews, S.R. Kulkarni, L. Panitzsch, S. Boden, S.I. Böttcher, I. Cernuda, R. Elftmann, F. Espinosa Lara, R. Gómez-Herrero, C. Terasa, J. Almena, S. Begley, E. Böhm, J.J. Blanco, W. Boogaerts, A. Carrasco, R. Castillo, A. Da Silva Fariña, V. De Manuel González, C. Drews, A.R. Dupont, S. Eldrum, C. Gordillo, O. Gutiérrez, D.K. Haggerty, J.R. Hayes, B. Heber, M.E. Hill, M. Jüngling, S. Kerem, V. Knierim, J. Köhler, S. Kolbe, A. Kulemzin, D. Lario, W.J. Lees, S. Liang, A. Martínez Hellín, D. Meziat, A. Montalvo, K.S. Nelson, P. Parra, R. Paspirgilis, A. Ravanbakhsh, M. Richards, O. Rodríguez-Polo, A. Russu, I. Sánchez, C.E. Schlemm, B. Schuster, L. Seimetz, J. Steinhagen, J. Tammen, K. Tyagi, T. Varela, M. Yedla, J. Yu, N. Agueda, A. Aran, T.S. Horbury, B. Klecker, K.-L. Klein, E. Kontar, S. Krucker, M. Maksimovic, O. Malandraki, C.J. Owen, D. Pacheco, B. Sanahuja, R. Vainio, J.J. Connell, S. Dalla, W. Dröge, O. Gevin, N. Gopalswamy, Y.Y. Kartavykh, K. Kudela, O. Limousin, P. Makela, G. Mann, H. Önel, A. Posner, J.M. Ryan, J. Soucek, S. Hofmeister, N. Vilmer, A.P. Walsh, L. Wang, M.E. Wiedenbeck, K. Wirth, Q. Zong, The energetic particle detector: Energetic particle instrument suite for the solar orbiter mission, Astron. Astrophys. 642 (2020) A7, http://dx.doi.org/10.1051/0004-6361/201935287, URL https://www.aanda.org/10.1051/0004-6361/201935287.

[32] M. Prieto, A. Ravanbakhsh, Ó. Gutiérrez, A. Montalvo, R.F. Wimmer-Schweingruber, G. Mason, I. Cernuda, F. Espinosa Lara, A. Carrasco, C. Martín, L. Seimetz, S.R. Kulkarni, L. Panitzsch, J.-C. Terasa, B. Schuster, M. Yedla, V. Knierim, S.I. Böttcher, S. Boden, R. Elftmann, N. Janitzek, B. Andrews, G. Ho, Ó. R-Polo, A. Martínez, R. Gómez-Herrero, S. Sánchez, J. Rodríguez-Pacheco, In-flight verification of the engineering design data for the energetic particle detector on board the ESA/NASA solar orbiter, Acta Astronaut. 187 (2021) 12–23, http://dx.doi.org/10.1016/j.actaastro.2021.06.007, URL https://www.sciencedirect.com/science/article/pii/S0094576521003040.

[33] Gaisler, L3STAT - LEON3 Statistics Unit, Tech. Rep., Cobham Gaisler, Goteburg, Sweden, 2023, pp. 1454–1462, URL https://web.archive.org/web/20241220202415if_/https://download.gaisler.com/products/GRLIB/doc/grip.pdf#G91.966345.

[34] Gaisler, LEON4 Statistics Unit, Tech. Rep., Cobham Gaisler, Goteburg, Sweden, 2015, pp. 1,324–328, URL https://web.archive.org/web/20220419233438/https://www.gaisler.com/doc/LEON4-N2X-DS.pdf#G31.966345.

[35] Gaisler, AHB Statistics Unit, Tech. Rep., Cobham Gaisler, Goteburg, Sweden, 2015, pp. 117–118, URL https://web.archive.org/web/20220129024413/http://microelectronics.esa.int/gr740/LEON4-NGMP-DRAFT-2-1.pdf#G10.1094073.

[36] S. Nolting, et al., The NEORV32 RISC-v processor: Datasheet, section 3.8.7, instrunction retired counter increment, 2024, URL https://web.archive.org/web/20231208221404/https://stnolting.github.io/neorv32/#_machine_counter_and_timer_csrs.

[37] N. Ho, P. Kaufmann, M. Platzner, Towards self-adaptive caches: A run-time reconfigurable multi-core infrastructure, in: 2014 IEEE International Conference on Evolvable Systems, 2014, pp. 31–37, http://dx.doi.org/10.1109/ICES.2014.7008719.

[38] D. Bakhvalov, Performance Analysis and Tuning on Moder CPUs, Tech. Rep., EasyPerf, 2020, p. 46, URL https://web.archive.org/web/20230531103427/https://faculty.cs.niu.edu/~winans/notes/patmc.pdf#subsection.4.1.

[39] T. Marena, RISC-V: High Performance Embedded SweRV™: Core Microarchitecture, Performance and CHIPS Alliance, Tech. Rep., Western Digital Corporation, 2019, URL https://web.archive.org/web/20240322111430/https://riscv.org/wp-content/uploads/2019/04/RISC-V_SweRV_Roadshow-.pdf#page=7.

[40] M. Prieto, D. Guzman, D. Meziat, S. Sanchez, L. Planche, LEON2 cache characterization. A contribution to WCET determination, in: 2007 IEEE International Symposium on Intelligent Signal Processing, 2007, pp. 1–6, http://dx.doi.org/10.1109/WISP.2007.4447578.

[41] W. Zhang, M. Lv, W. Chang, L. Ju, Precise and scalable shared cache contention analysis for WCET estimation, in: Proceedings of the 59th ACM/IEEE Design Automation Conference, DAC '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 1267–1272, http://dx.doi.org/10.1145/3489517.3530613, URL https://dl.acm.org/doi/10.1145/3489517.3530613.

[42] M.E. Thomadakis, The Architecture of the Nehalem Processor and Nehalem-EP SMP Platforms, Tech. Rep., Texas A&M University, 2011, p. 12, URL https://web.archive.org/web/20230126162244/https://courses.cs.washington.edu/courses/cse470/19sp/nehalem.pdf#page=12.

[43] R. Farber, Chapter 4 - the CUDA execution model, in: CUDA Application Design and Development, Morgan Kaufmann, Boston, 2011, pp. 85–108, http://dx.doi.org/10.1016/B978-0-12-388426-8.00004-5, URL https://www.sciencedirect.com/science/article/pii/B9780123884268000045.

[44] R. Pellizzoni, E. Betti, S. Bak, G. Yao, J. Criswell, M. Caccamo, R. Kegley, A predictable execution model for COTS-based embedded systems, in: 2011 17th IEEE Real-Time and Embedded Technology and Applications Symposium, (ISSN: 1545-3421) 2011, pp. 269–279, http://dx.doi.org/10.1109/RTAS.2011.33.

[45] S. Nolting, et al., The NEORV32 RISC-V processor, 2022, URL https://web.archive.org/web/20230413220056/https://github.com/stnolting/neorv32.

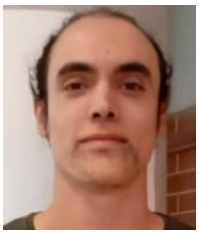
[46] GCC, RISC-V GNU compiler toolchain, 2023, URL https://web.archive.org/web/20230515073157/https://github.com/riscv-collab/riscv-gnu-toolchain.

[47] AMD, Xilinx vivado design suite: UG973 (v2018.3), 2018, URL https://web.archive.org/web/20230526075927/https://www.xilinx.com/products/design-tools/vivado.html.

[48] Digilent, Nexys 4 DDR Reference Manual, Tech. Rep., Digilent, 2016, URL https://web.archive.org/web/20230531192224/https://digilent.com/reference/_media/reference/programmable-logic/nexys-4-ddr/nexys4ddr_rm.pdf.

[49] J. Bennett, P. Dabbelt, C. Garlati, G. Madhusudan, T. Mudge, D. Patterson, Embench: An evolving benchmark suite for embedded iot computers from an academic-industrial cooperative, 2019, URL https://web.archive.org/web/20221207104221/https://riscv.org/wp-content/uploads/2019/06/9.25-Embench-RISC-V-Workshop-Patterson-v3.pdf.

[50] S. Nolting, et al., The NEORV32 RISC-V processor: Datasheet, section 1.1, a multi-cycle architecture?!?!, 2024, URL https://web.archive.org/web/20240402185911/https://stnolting.github.io/neorv32/#_rationale.



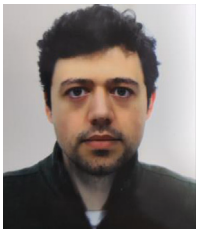
Miguel Jiménez Arribas obtained his M.Sc. degree in Space Science and Technology from the University of Alcalá (UAH), Spain, in 2019, and is currently pursuing his Ph.D. in Space Research and Astrobiology at the same institution. Since 2019, he has been an active member of the Department of Computer Engineering and the Space Research Group (SRG) within UAH, where he has contributed to various projects centered around the optimization of the RISC-V architecture for space applications. His research interests include computer architecture, onboard satellite hardware and software, and real-time embedded systems, reflecting his passion for advancing technology in space exploration.



Agustín Martínez received the M.S. degree from the Universidad Politécnica de Madrid (UPM) in 1986 and the Ph.D. degree from the Universidad de Alcalá in 2001. Professor of Computer Engineering Department at the University of Alcalá in Spain, Head of Department from 2010 to 2016. His research and teaching activities are in the areas of DSP, Computer Architecture, Embedded Systems and Space Systems. He has been in charge of technical and management issues at Telettra, Alcatel and Alcatel-Lucent over more than 20 years, accounting a deep experience over technical and management leadership activities in international projects.



Dr. Manuel Prieto Mateo is currently associate professor in the University of Alcalá, Spain. He obtained his degree in Electronics Engineering in 1999 and his Ph.D. in 2005. His research interests are digital electronics development based on FPGAs and radio-communications. He has led several national and international research projects in the field of space hardware and software development. He was the Project Manager of the Energetic Particle Detector (EPD) instrument on board the ESA/NASA Solar Orbiter mission. Dr. Prieto is also involved in research projects in Antarctica for the automation of measurement stations.



Iván Gamino del Río Dedicated Ph.D. student in Space Research with over 5 years of expertise in VHDL FPGA Design, low-level programming, and system behavior analysis. Proficient in designing, developing, and testing customized RISC-V processing cores, memory, and bus interfaces. Possesses a keen interest in exploring diverse architectures, including SPARC (LEON family), RISC-V (NOEL-V), and ARM (Cortex family). Committed to ongoing knowledge enhancement to excel as a Computer Engineer and become an influential educator in the field.



Andrea Fernández Gallego is a recent graduate of the Master's degree program in Science and Technology from Space. During her studies, she actively contributed to research as a member of the Space Research Group at Alcalá University (UAH), where she played a significant role in assisting with the development of this paper. With a strong foundation in space science and technology, Andrea now applies her expertise as an FPGA design and verification engineer, where she continues to contribute to cutting-edge projects in the field.



Óscar R. Polo received his M.S. degree in physics from the Universidad del Pais Vasco in 1994 and his Ph.D. in physics from the Universidad Complutense de Madrid, Spain, in 2003. Since 2004, he has been with the Computing Engineering Department at the University de Alcalá. He is currently an assistant professor of embedded and real-time systems. His research interests include computer architecture, satellite on-board software, model driven engineering and embedded real-time systems. He has actively participated in several research projects in the area of computer engineering of satellite platforms in NANOSAT-01, NANOSAT-1B, Solar Orbiter and Euclid missions.



Antonio Da Silva Born in Germany, Antonio da Silva has received the M.Sc. and Ph.D. degrees in computer engineering from Universidad de Alcalá (UAH), in 2001 and 2015, respectively. He has worked as an embedded software developer for primary radar systems and since 1991 he lectures in the field of Computing Science. He has worked with the Space Research Group of Universidad de Alcalá on projects related to the development of the energetic particle detector (EPD) on-board Solar Orbiter. His research interests include Serious Games for Education, Fault Tolerant systems design and critical software early development and verification.



Pablo Parra received his Ph.D. in Information and Communication Technologies from the University of Alcalá in 2012. Since 2006, he has been working with the Computer Engineering Department and the Space Research Group (SRG) of the University of Alcalá. His research interests include component-based software engineering and model-driven engineering applied to the field of real-time embedded systems. He has taken part in numerous research projects in the field of on-board satellite software development, such as the NANOSAT programme, Solar Orbiter and Euclid.



Sebastián Sánchez received his M.S. degree from the Universidad Politécnica de Madrid in 1994 and his PhD. at the Universidad de Alcalá, in 1998. Since 1990, he has been with the Computing Engineering Department at the University de Alcalá. He is full professor of operating systems and computer architecture. His research interests include space instrumentation, embedded realtime systems, and mobile robots. He has actively participated in several national and international research projects in the area of hardware and software on board satellites such as SOHO, PHOTON, FUEGO 2, NANOSAT, Exomars, MICROSAT, Solar Orbiter and Euclid.